

Task 4: Exploitation & System Security

1. Introduction

Exploitation and System Security are critical components of cybersecurity that focus on identifying, exploiting, and mitigating vulnerabilities in computer systems. This task aims to provide hands-on understanding of penetration testing workflows, controlled exploitation techniques, and system hardening practices. The objective is to simulate real-world attack scenarios ethically and to learn how to defend systems against potential threats.

This task follows an ethical hacking approach, where all activities are performed in a controlled lab environment for educational purposes only.

2. Penetration Testing Methodology

Penetration testing is a structured process used to evaluate the security of a system by simulating attacks.

2.1 Phases of Penetration Testing

1. Reconnaissance (Recon)

This phase involves gathering information about the target system, such as IP addresses, open ports, services, and system architecture.

2. Scanning

In this phase, tools are used to identify vulnerabilities, running services, and possible entry points in the system.

3. Exploitation

Exploitation involves leveraging discovered vulnerabilities to gain access to the system in a controlled manner.

4. Post-Exploitation

After gaining access, further analysis is done to understand system privileges, data exposure, and persistence risks.

5. Reporting

All findings, vulnerabilities, and mitigation strategies are documented clearly in a penetration testing report.

3. Exploitation Using Metasploit

Metasploit is a widely used penetration testing framework that helps security professionals test system vulnerabilities.

3.1 Purpose of Metasploit

Identify known vulnerabilities

Exploit vulnerable services

Test system security posture

Demonstrate real-world attack scenarios

3.2 Exploitation Concept

A vulnerable machine (such as a deliberately vulnerable lab setup) is tested using Metasploit modules to simulate unauthorized access. Post-exploitation analysis helps understand system information and security gaps.

4. Password Attacks

Password attacks target weak authentication mechanisms and poor password policies.

4.1 Brute Force Attacks

Brute force attacks involve systematically trying multiple password combinations until the correct one is found. Weak passwords and lack of account lockout policies increase risk.

4.2 Hash Cracking

Passwords stored in hashed form can sometimes be cracked if weak hashing algorithms or simple passwords are used.

4.3 Security Risks

Weak passwords

Reused credentials

Lack of strong password policies

5. Social Engineering (Simulation Only)

Social engineering exploits human psychology rather than technical vulnerabilities.

5.1 Phishing Simulation

A phishing simulation demonstrates how attackers trick users into revealing sensitive information using fake emails or login pages.

5.2 Awareness Training

Users are trained to identify:

Suspicious links

Fake email senders

Urgent or threatening messages

This task focuses on awareness and prevention, not real attacks.

6. Malware Basics

Malware refers to malicious software designed to harm systems or steal data.

6.1 Types of Malware

Viruses

Worms

Trojans

Ransomware

6.2 Malware Analysis

Static Analysis: Examining malware without execution

Dynamic Analysis: Observing malware behavior in a sandbox environment

Only benign samples are analyzed for educational purposes.

7. System Hardening

System hardening is the process of securing a system by reducing its attack surface.

7.1 Hardening Techniques

Applying security patches

Configuring firewalls

Disabling unnecessary services

Implementing strong password policies

7.2 Importance

Proper system hardening significantly reduces the risk of exploitation and unauthorized access.

8. Security Mitigations and Best Practices

Use strong and unique passwords

Enable firewalls and intrusion detection systems

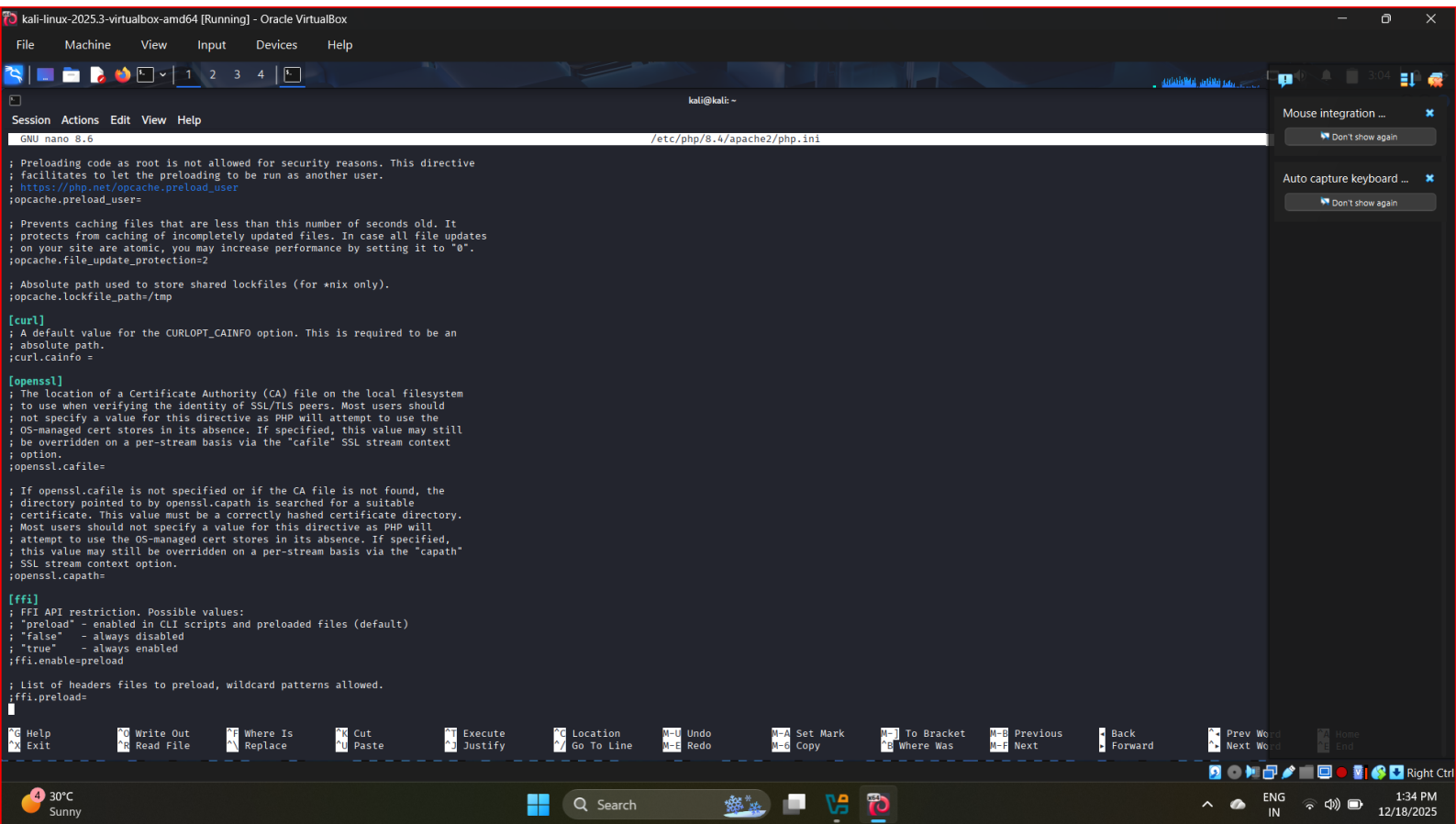
Regularly update software

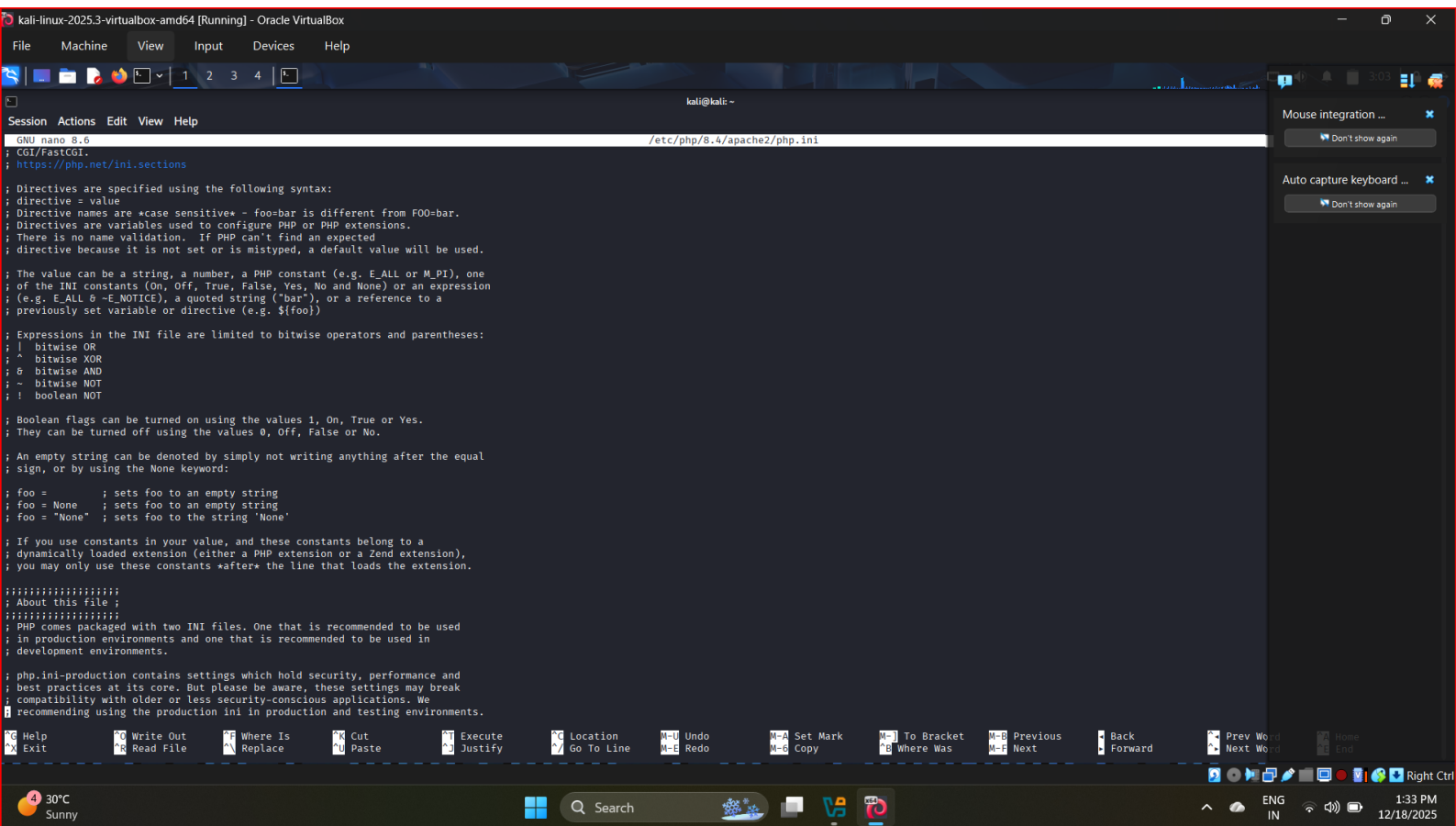
Conduct periodic security audits

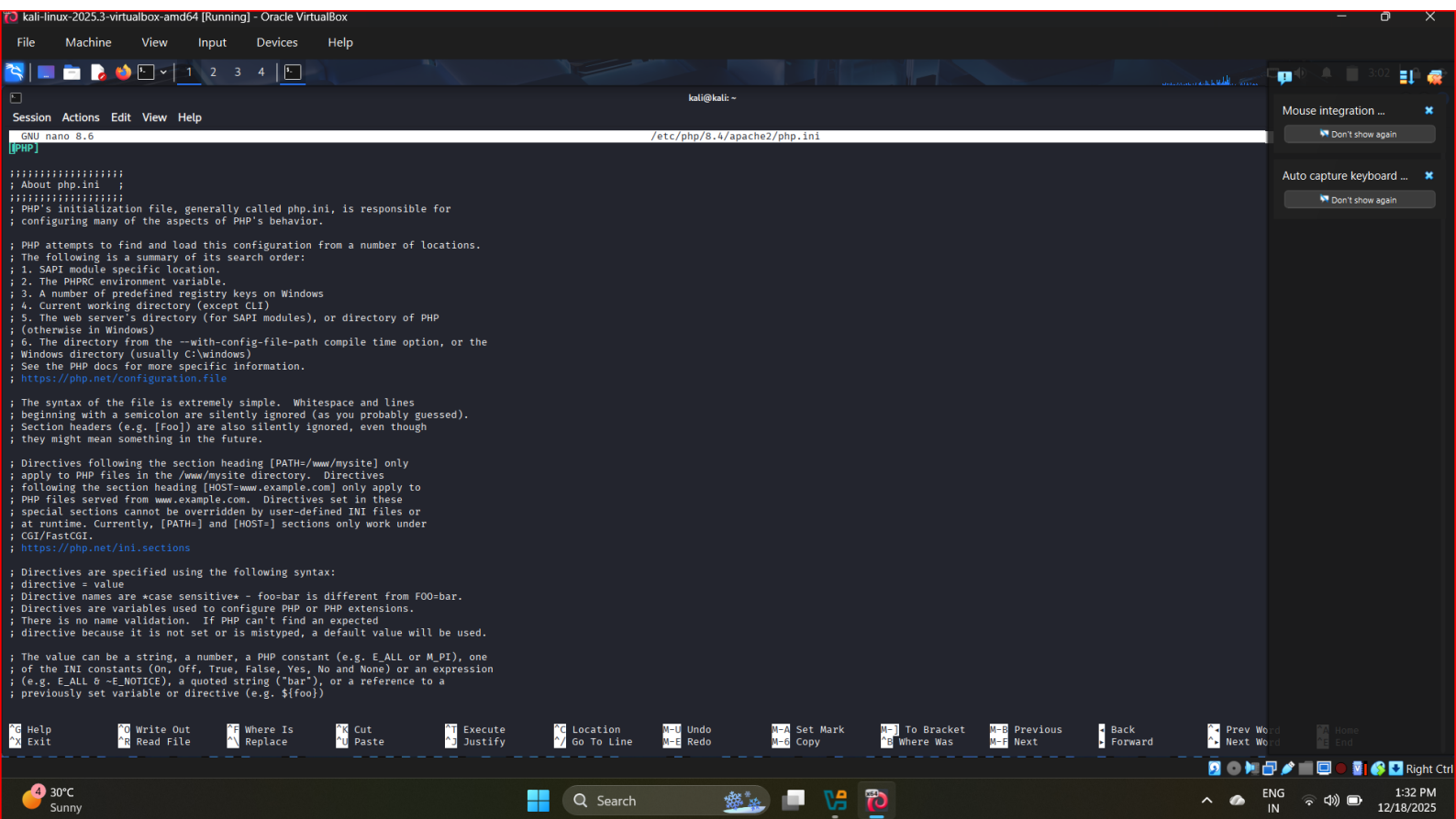
Train users against social engineering attacks

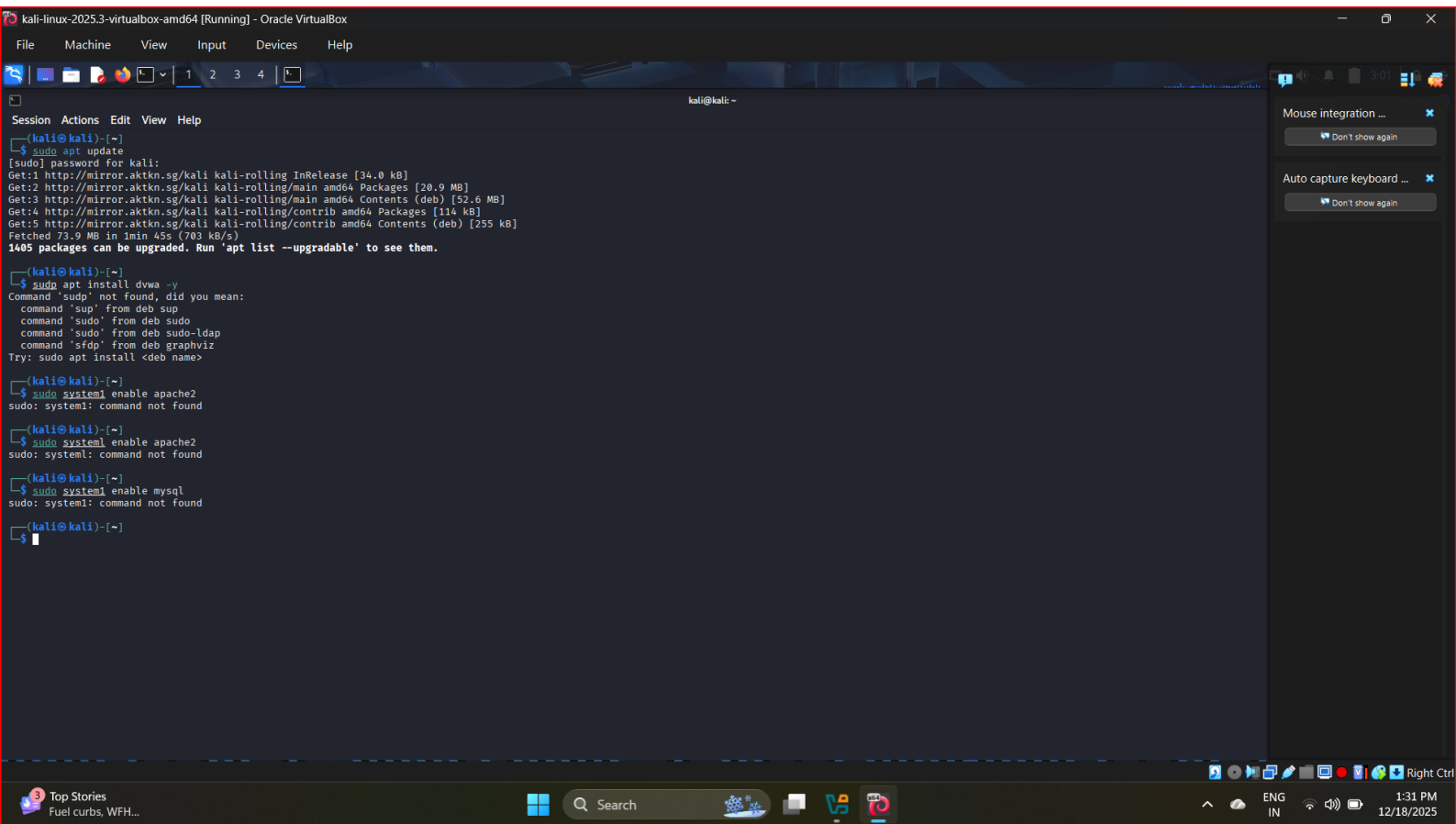
9. Conclusion

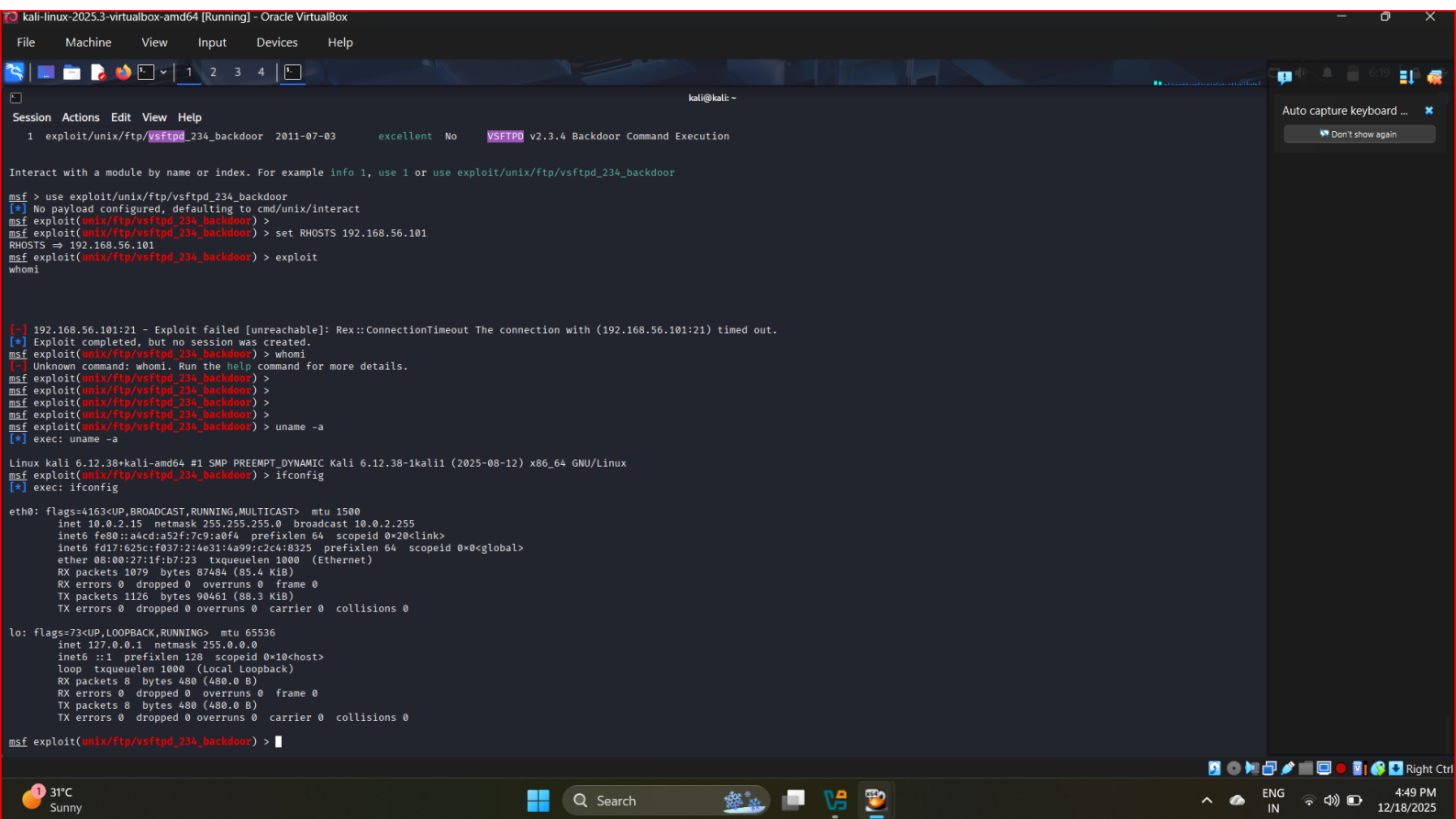
This task provided comprehensive exposure to exploitation techniques and defensive security measures. By understanding how attacks occur, better preventive strategies can be implemented. Ethical hacking practices play a vital role in strengthening system security and protecting sensitive data.

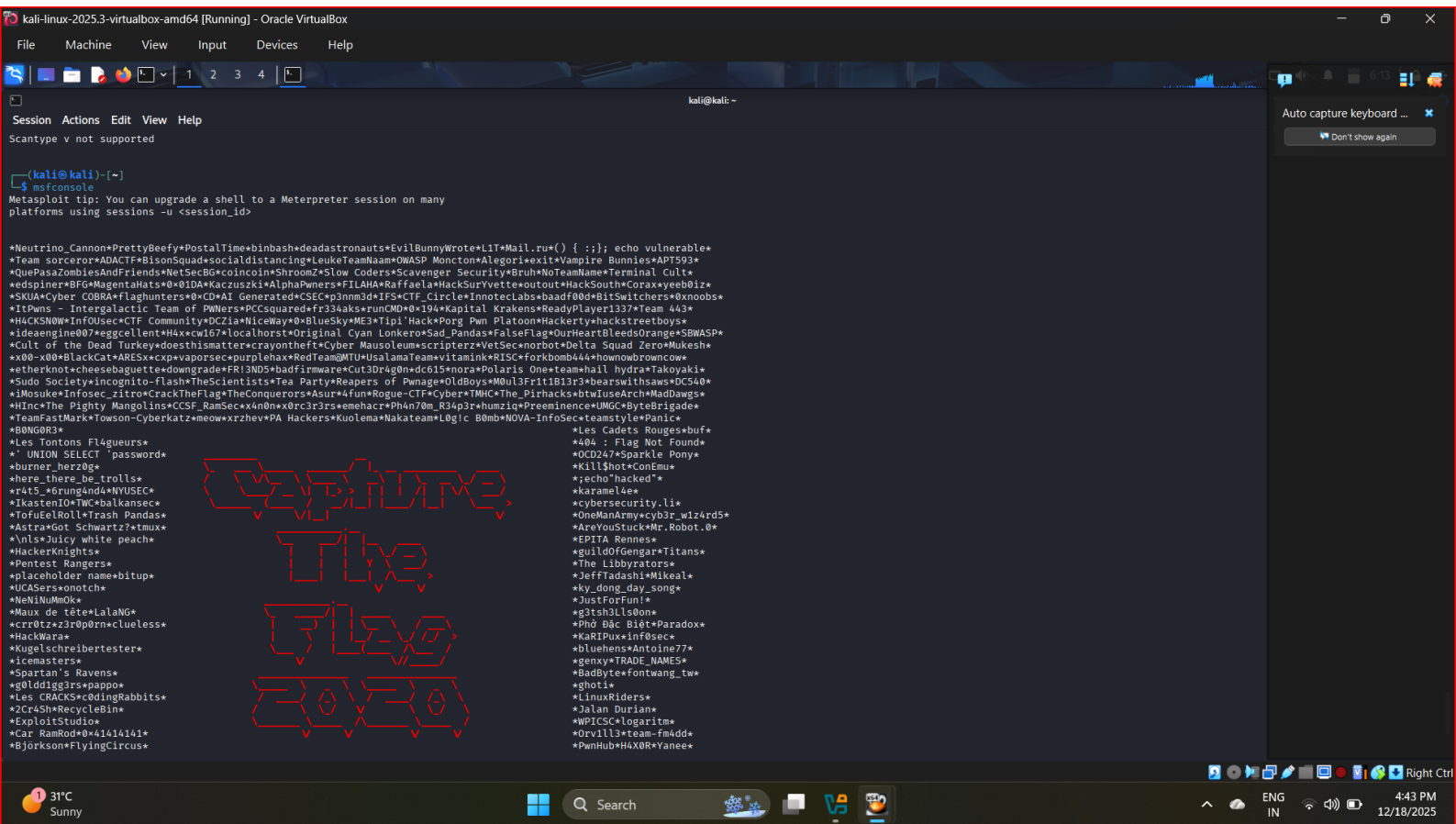


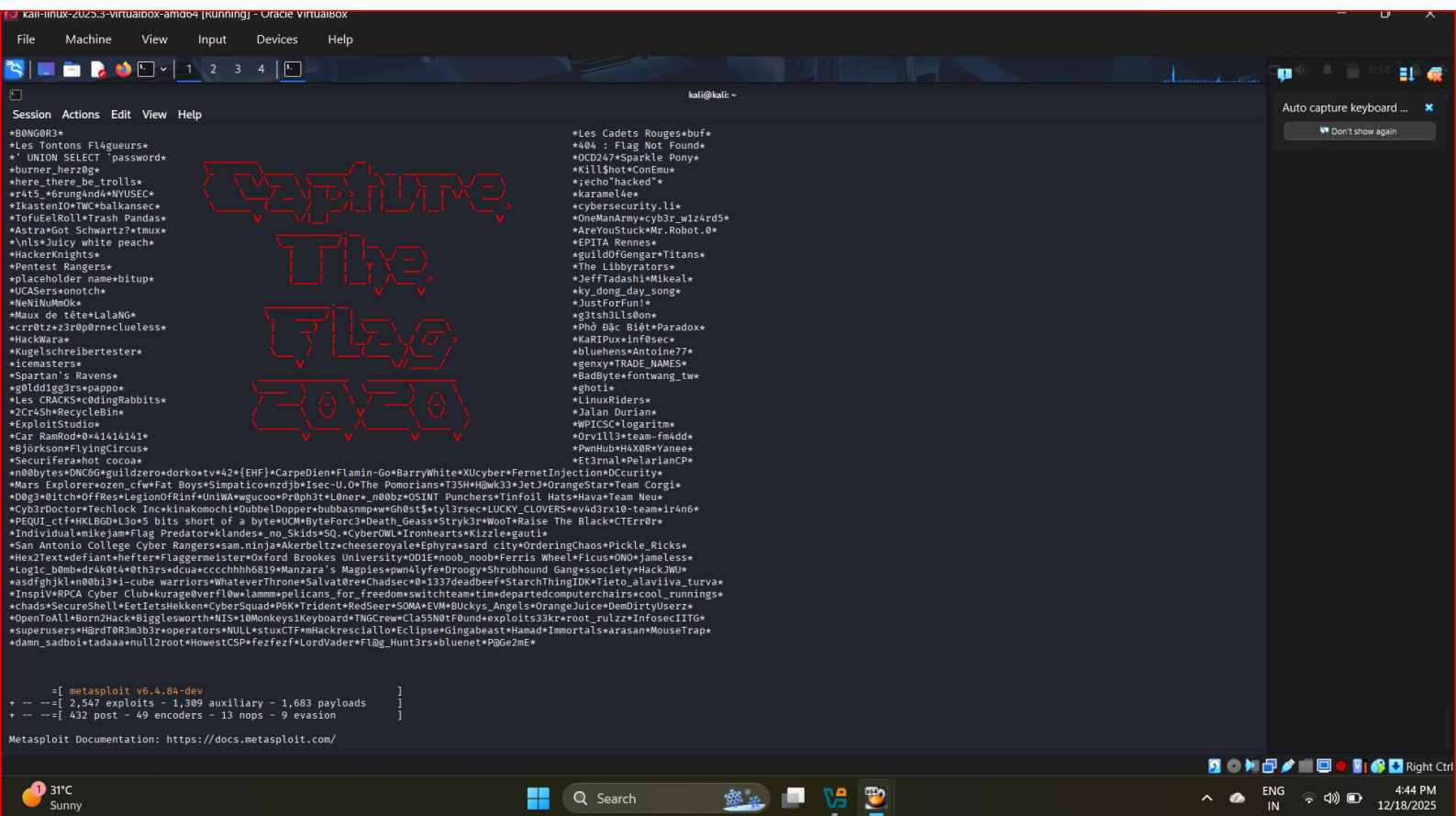












kali-linux-2025.3-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1 2 3 4

kali@kali: ~

Session Actions Edit View Help

```
--min-rate <number>: Send packets no slower than <number> per second
--max-rate <number>: Send packets no faster than <number> per second
FIREWALL/IDS EVASION AND SPOOFING:
-f; --mtu <val>: fragment packets (optionally w/given MTU)
-D <decoy1,decoy2[,ME],...>: Cloak a scan with decoys
-S <IP_Address>: Spoof source address
-e <iface>: Use specified interface
-g/--source-port <portnum>: Use given port number
--proxies <url1,[url2],...>: Relay connections through HTTP/SOCKS4 proxies
--data <hex string>: Append a custom payload to sent packets
--data-string <string>: Append a custom ASCII string to sent packets
--data-length <num>: Append random data to sent packets
--ip-options <options>: Send packets with specified ip options
--ttl <val>: Set IP time-to-live field
--spoof-mac <mac address/prefix/vendor name>: Spoof your MAC address
--badsum: Send packets with a bogus TCP/UDP/SCTP checksum
OUTPUT:
-oN/-oX/-oS/-oG <file>: Output scan in normal, XML, s|<rIpt kIddi3,
and Grepable format, respectively, to the given filename.
-oA <basename>: Output in the three major formats at once
-v: Increase verbosity level (use -vv or more for greater effect)
-d: Increase debugging level (use -dd or more for greater effect)
--reason: Display the reason a port is in a particular state
--open: Only show open (or possibly open) ports
--packet-trace: Show all packets sent and received
--iflist: Print host interfaces and routes (for debugging)
--append-output: Append to rather than clobber specified output files
--resume <filename>: Resume an aborted scan
--noninteractive: Disable runtime interactions via keyboard
--stylesheet <path/URL>: XSL stylesheet to transform XML output to HTML
--webxml: Reference stylesheet from Nmap.Org for more portable XML
--no-stylesheet: Prevent associating of XSL stylesheet w/XML output
ISC:
-6: Enable IPv6 scanning
-A: Enable OS detection, version detection, script scanning, and traceroute
--datadir <dirname>: Specify custom Nmap data file location
--send-eth/--send-ip: Send using raw ethernet frames or IP packets
--privileged: Assume that the user is fully privileged
--unprivileged: Assume the user lacks raw socket privileges
-V: Print version number
-h: Print this help summary page.
EXAMPLES:
nmap -v -A scanme.nmap.org
nmap -v -sn 192.168.0.0/16 10.0.0.0/8
nmap -v -iR 10000 -Pn -p 80
SEE THE MAN PAGE (https://nmap.org/book/man.html) FOR MORE OPTIONS AND EXAMPLES
cantype v not supported
```

~(kali@kali)-[~]

\$

31°C
Sunny



Search



ENG
IN

Session Actions Edit View Help

Nmap scan report for 192.168.56.255

Host is up (0.0099s latency).

Nmap done: 256 IP addresses (256 hosts up) scanned in 162.66 seconds

(kali@kali)-[~]

\$ nmap -sv 192.168.56.101

Nmap 7.95 (<https://nmap.org>)

Usage: nmap [Scan Type(s)] [Options] {target specification}

TARGET SPECIFICATION:

Can pass hostnames, IP addresses, networks, etc.

Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254

-iL <inputfilename>: Input from list of hosts/networks

-iR <num hosts>: Choose random targets

--exclude <host1[,host2][,host3],...>: Exclude hosts/networks

--excludefile <exclude_file>: Exclude list from file

HOST DISCOVERY:

-sL: List Scan - simply list targets to scan

-sn: Ping Scan - disable port scan

-Pn: Treat all hosts as online -- skip host discovery

-PS/PA/PU/PY[portlist]: TCP SYN, TCP ACK, UDP or SCTP discovery to given ports

-PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes

-PO[protocol list]: IP Protocol Ping

-n/-R: Never do DNS resolution/Always resolve [default: sometimes]

--dns-servers <serv1[,serv2],...>: Specify custom DNS servers

--system-dns: Use OS's DNS resolver

--traceroute: Trace hop path to each host

SCAN TECHNIQUES:

-sS/sT/sA/sW/sM: TCP SYN/Connect()/ACK/Window/Maimon scans

-sU: UDP Scan

-sN/sF/sX: TCP Null, FIN, and Xmas scans

--scanflags <flags>: Customize TCP scan flags

-sI <zombie host[:probeport]>: Idle scan

-sY/sZ: SCTP INIT/COOKIE-ECHO scans

-sO: IP protocol scan

-b <FTP relay host>: FTP bounce scan

PORT SPECIFICATION AND SCAN ORDER:

-p <port ranges>: Only scan specified ports

Ex: -p22; -p1-65535; -p U:53,111,137,T:21-25,80,139,8080,S:9

--exclude-ports <port ranges>: Exclude the specified ports from scanning

-F: Fast mode - Scan fewer ports than the default scan

-r: Scan ports sequentially - don't randomize

--top-ports <number>: Scan <number> most common ports

--port-ratio <ratio>: Scan ports more common than <ratio>

SERVICE/VERSION DETECTION:

-sV: Probe open ports to determine service/version info

--version-intensity <level>: Set from 0 (light) to 9 (try all probes)

--version-light: Limit to most likely probes (intensity 2)

--version-all: Try every single probe (intensity 9)

--version-trace: Show detailed version scan activity (for debugging)

SCRIPT SCAN:

-sC: equivalent to --script=default

